Performance Testing

Date	14 July, 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	**Details**
Testing Tool Used	Postman, Browser (Chrome DevTools)
Type of Testing	Functional Testing, UI Testing, Prediction Accuracy Testing
Target Module	ExpressJS API (/predict), CreditAI Web Interface
Test Environment	Local System (Windows 11, Python 3.11, ExpressJS/Node.js)
Test Date	14 July, 2026

Step 2: Test Scenarios

S.No	**Test Scenario / Description**	**No. of virtual Users**	**Duration (sec)**	**Expected Outcome**
1	Single user submits credit card application via CreditAI UI	1	30	Prediction generated within 500ms with confidence % shown
2	Continuous prediction requests	50	60	Stable response time with no crashes
3	Peak load testing of ML production engine.	100	120	Application remains responsive and maintains acceptable performance

Step 3: Performance Test Results

S.No	**Metric**	**Target Value**	**Actual Value**	**Status (pass/fail)**	**Remarks**
1	Response Time (Avg)	< 2 seconds	<500m sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	3.63 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	>40 req/sec	52 req/sec	Pass	Good request handling capacity
4	Error Rate	<1%	0%	Pass	No request failures
5	CPU Utilization	<80%	61%	Pass	Efficient CPU usage
6	Memory Utilization	<80%	57%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- The Credit Card Approval Prediction system successfully processed concurrent user requests.
- Average prediction response time remained below 500 milli seconds.
- No failed prediction requests were observed during testing.
- The ExpressJS backend and Python ML engine remained stable under moderate and heavy workloads.
- The Random Forest model delivered highly accurate predictions (88.41%)

Bottlenecks Identified

- Minor increase in response time when more than 100 concurrent users accessed the application simultaneously.
- Initial model loading into memory via joblib caused a slight delay during first rqust

Optimization Steps Taken

- Optimized ExpressJS routing and client-server request handling.
- Reduced unnecessary preprocessing by only encoding specific categorical columns (Gender, Industry, Citizen).
- Loaded the trained Random Forest model once during application startup rather than on each request.

Step 5: Screenshots / Evidence